Containerize the Weights: Simple, Lightning-Fast Model Deployment on NVIDIA DGX Clusters

Container images + peer-to-peer distribution: 13 minutes → 3 seconds per replica

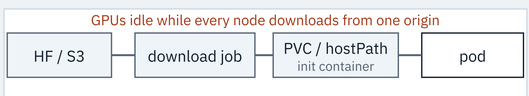
Omer Ozdemir · Burak Gulhan

1. Objective

Make large-model deployment simple and fast by treating model weights like application code:

- **Package** weights as container images, mounted with Kubernetes Image Volumes
- **Distribute** them peer-to-peer across the cluster fabric instead of from a central source
- **Optimize** every layer of the container stack to run at network or disk speed

2. Challenges



- Deploying large models on GPU clusters has two problems: it is **complex**, and it is **slow**.
- As frontier models grow to **terabytes** (*Qwen3.8 at 2.5 TB*), popular serving stacks, including vLLM, SGLang, and NVIDIA Dynamo, have layered on **workarounds** to keep up: runtime downloads, init containers, shared volumes, pre-staged host paths, and dedicated download jobs.
- Each layer is **one more thing to operate, secure, and debug**, and weights live outside the deployment, with **no versioning or clean rollback** alongside the code that serves them.
- Furthermore, deployment remains slow: ***GPUs sit idle*** while weights download.
- The orchestration layer and scheduler **don't know where model weights are cached** locally, so they can't take that into account when scheduling workloads.

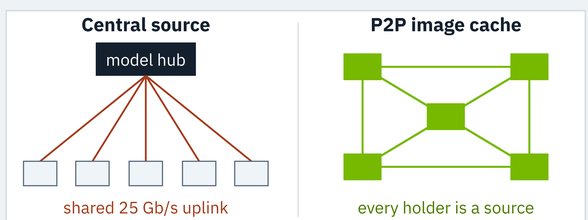
3. Our Solution

3.1 Weights as container images

- **We take a simpler approach.**
- We package model weights as **container images** and mount them into pods with ***Kubernetes Image Volumes***.
- The model is then part of the deployment and is **versioned and rolled back just like application code**, through Kubernetes.
- The container runtime also **caches the image on each node**, so a restarted or rescheduled pod finds its weights *already there*.

```
volumes:
- name: model
  image:
    reference: registry/qwen3.8@sha256:…
containers:
- volumeMounts:
  - {name: model, mountPath: /models}
```

3.2 Peer-to-peer distribution



- Containerized weights make **peer-to-peer image caching** straightforward, so we can *put that bandwidth to work*.
- **Every node that holds a model becomes a source**, so distribution **scales with the cluster**.

3.3 Using DGX B300 Fabric Bandwidth

- Today, nodes typically download weights from a central source whose uplink, around **25 Gb/s**, is *shared by the nodes in the cluster*.
- At that rate, a 2.5 TB model takes **over 13 minutes per replica**, with its *GPUs idle the whole time*.
- Meanwhile, each NVIDIA DGX B300 node has **6.4 Tb/s** of fabric bandwidth *sitting unused*.

~~25 Gb/s~~ → **6.4 Tb/s**

13 min	**~3 s**
central source, GPUs idle the whole time	peer-to-peer at fabric speed

- Nodes pull weights from each other at **up to 256× the central uplink**, reducing download time from **13 minutes** to **about 3 seconds**.

3.4 Optimizing Every Layer

- **Reaching these speeds is not trivial.**
- Container tooling was designed for *megabyte- to gigabyte-sized images*, **not terabyte-scale weights**.
- We **identify the bottleneck at each layer**, from image build and publishing, registries, and P2P distribution to networking, kubelet, and containerd, and **show how to remove it**.

Build & publish	**Registry**	**P2P**
Network	**kubelet**	**containerd**

- With these optimizations, distribution runs at the **full speed of the network or local disk**, whichever is the bottleneck.

4. Key Achievements

- **Up to 256× faster** weight download than a 25 Gb/s central source
- 2.5 TB model per replica: **13 minutes → ~3 seconds**
- Distribution at the **full speed of the network or local disk**
- Models **versioned and rolled back** like application code
- **No extra infrastructure**: no download jobs, shared volumes or pre-staged host paths
- **Cached by default**: restarted and rescheduled pods find weights in place
- **Scales with the cluster**: every node is a source

References

1. Kubernetes Enhancement Proposal 4639, *OCI VolumeSource* (Image Volumes). github.com/kubernetes/enhancements
2. Spegel: stateless cluster-local OCI registry mirror. github.com/spegel-org/spegel
3. containerd and BuildKit. containerd.io; github.com/moby/buildkit
4. M. Kliukovkin, *Cold-Start Model Delivery in Kubernetes Inference Serving*, arXiv:2607.16596, 2026.
5. NVIDIA Dynamo documentation: model caching and ModelExpress. docs.nvidia.com/dynamo
6. vLLM and SGLang Kubernetes deployment guides. docs.vllm.ai; docs.sglang.ai
7. NVIDIA DGX B300 User Guide. docs.nvidia.com/dgx
8. S. Mukherjee, *Cold Start Mechanics: How Warm Pools and Model Caching Actually Work*, DigitalOcean, Oct. 2026.
9. Omer Oz, *The Hidden Critical Path: An Incremental Analysis of Container Image Pull Bottlenecks in containerd*, v1, Oct. 2026.